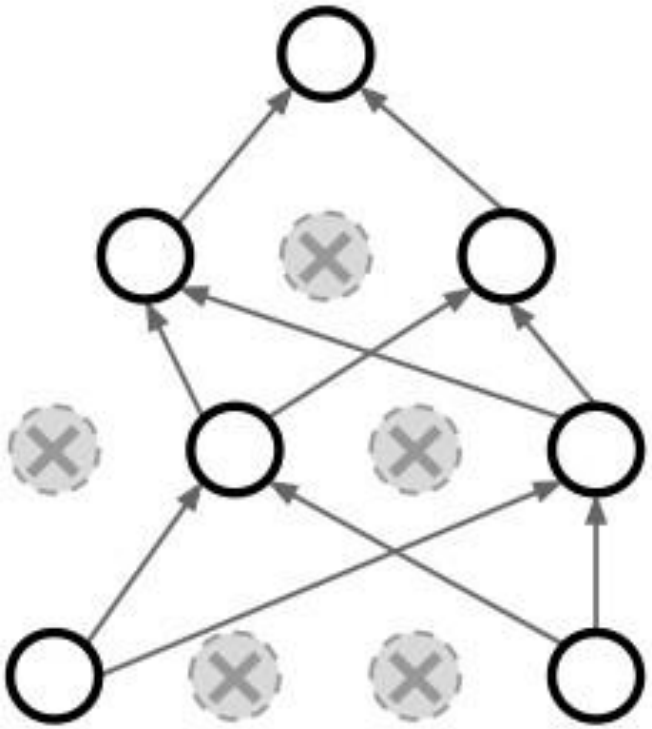
Regularization 2: Dropout

Motivation

- A good model should be robust under modest changes in the input
  - Training with input noise equal to Tikhonov Regularization
  - Dropout: inject noises into internal layers

Why does drop-out work?

How can this possibly be a good idea?



Forces the network to have a redundant representation;
Prevents co-adaptation of features

Add Noise without Bias

- Add noise into ***x*** to get ***x'***, we hope

$$\mathbf{E}[\mathbf{x}'] = \mathbf{x}$$

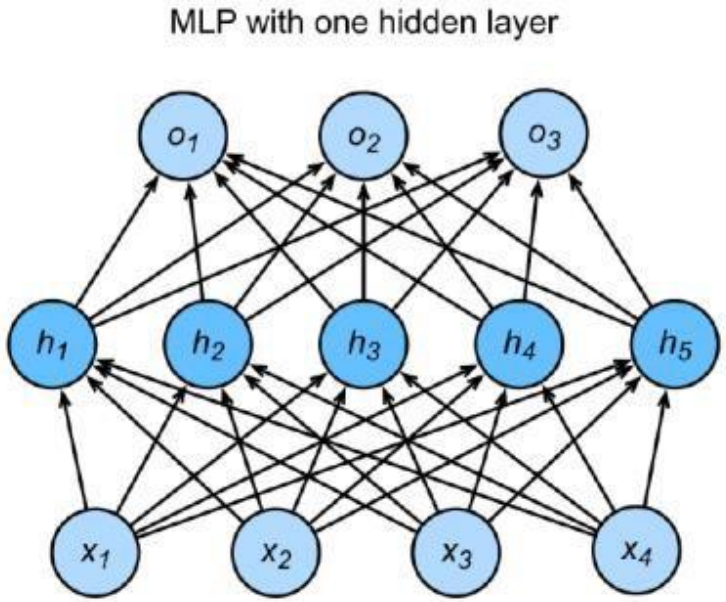
- Dropout perturbs each element by

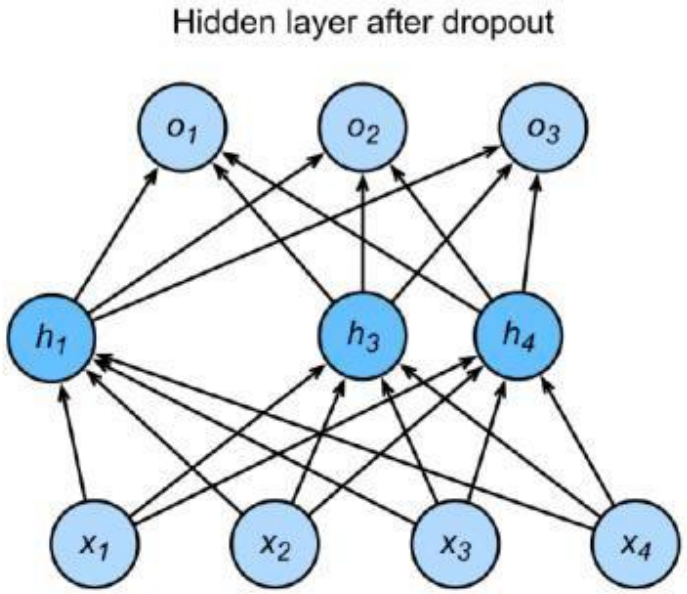
$$x_i' = \begin{cases} 0 & \text{with probablity } p \\ \dfrac{x_i}{1-p} & \text{otherise} \end{cases}$$

Apply Dropout

- Often apply dropout on the output of hidden fully-connected layers

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}' = \text{dropout}(\mathbf{h})$$
$$\mathbf{o} = \mathbf{W}_2\mathbf{h}' + \mathbf{b}_2$$
$$\mathbf{y} = \text{softmax}(o)$$



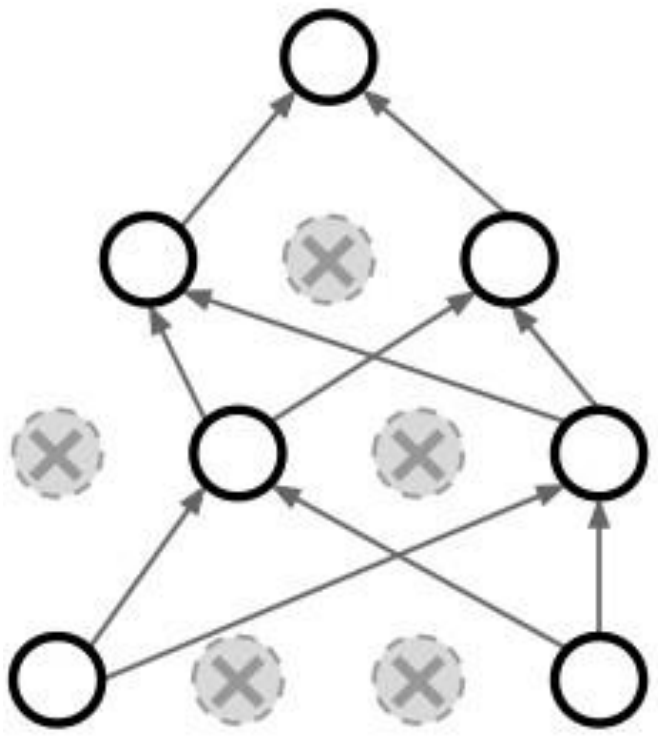


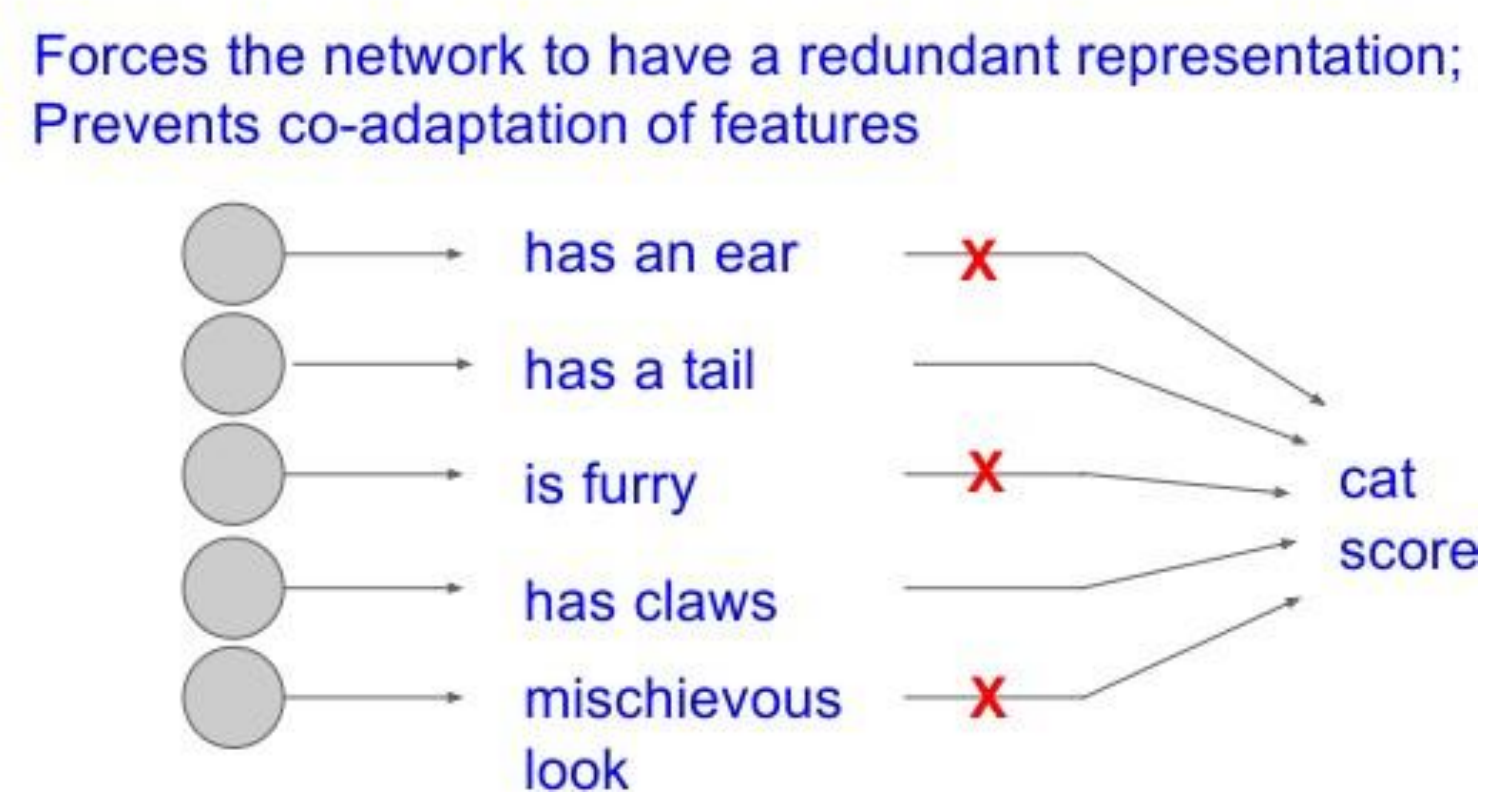
Dropout in inference

- Dropout is only used in training
- No node will be deleted when we use the model for prediction
  - Guarantee deterministic results

Why does drop-out work?

- Intuition: Can't rely on any one feature, so have to spread out weights.





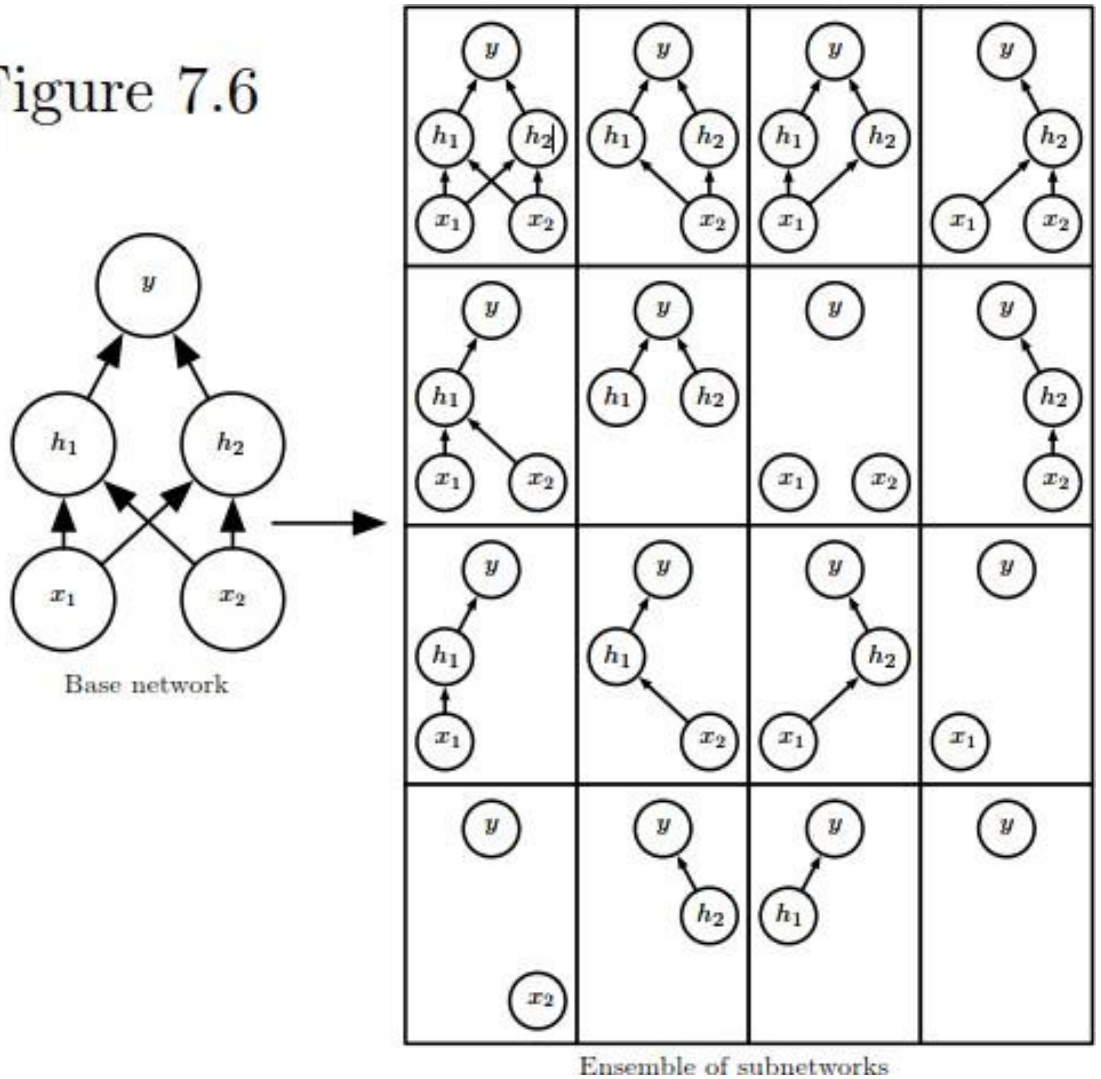
Why does drop-out work?

- Intuition: Can't rely on any one feature, so have to spread out weights.
- Often used in computer vision: learning noise is useless!

- Downside: less well defined obj function, won't decay as iteration increases

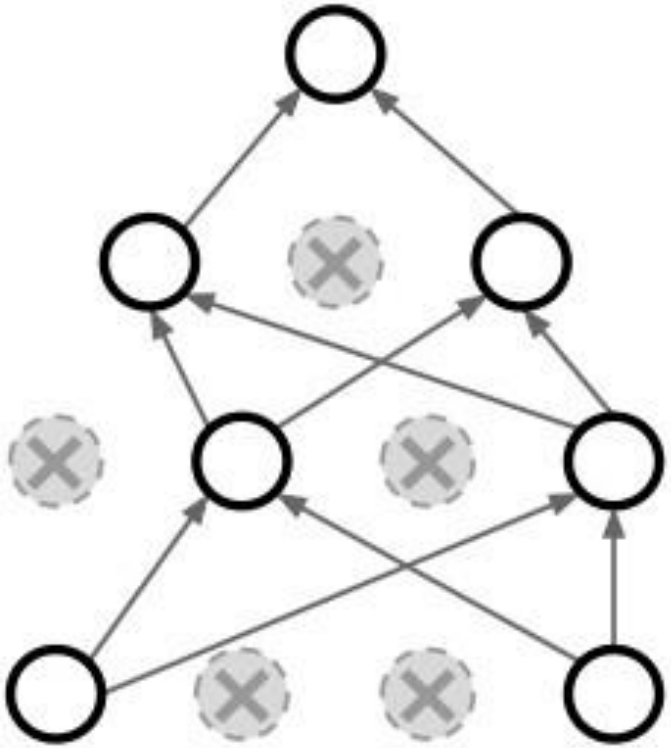
Why does drop-out work?

Figure 7.6



Why does drop-out work?

How can this possibly be a good idea?



Another interpretation:

Dropout is training a large **ensemble** of models (that share parameters).

Each binary mask is one model

An FC layer with 4096 units has $2^{4096} \sim 10^{1233}$ possible masks!
Only $\sim 10^{82}$ atoms in the universe...

Summary of regularization

- Weight decay uses L2 regularization to make the weight not so big, hence controlling the model complexity
- Regularization has hyper-parameter

- Dropout introduces 0 randomly to control model complexity
- Dropout is often applied to hidden layers
- Dropout rate is hyper-parameter

Numerical stability

Gradients for Neural Networks

- Consider a network with *d* layers

$$\mathbf{h}^t = f_t\left(\mathbf{h}^{t-1}\right) \text{ and } y = \ell \circ f_d \circ \cdots \circ f_1(\mathbf{x})$$

- How to compute the gradient? Chain rule from calculus: $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$
- Compute the gradient of the loss ℓ w.r.t. $\mathbf{W}_t$

$$\frac{\partial \ell}{\partial \mathbf{W}^t} = \frac{\partial \ell}{\partial \mathbf{h}^d} \underbrace{\frac{\partial \mathbf{h}^d}{\partial \mathbf{h}^{d-1}} \cdots \frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t}}_{\text{Multiplication of } d\text{-}t \text{ matrices}} \frac{\partial \mathbf{h}^t}{\partial \mathbf{W}^t}$$

Two Issues for Deep Neural Networks

$$\prod_{i=t}^{d-1} \frac{\partial \mathbf{h}^{i+1}}{\partial \mathbf{h}^{i}}$$

Gradient Exploding

$$1.5^{100} \approx 4 \times 10^{17}$$

Gradient Vanishing

$$0.8^{100} \approx 2 \times 10^{-10}$$

Example: MLP

- Assume MLP (without bias for simplicity)

$$f_t\left(\mathbf{h}^{t-1}\right) = \sigma\left(\mathbf{W}^t\mathbf{h}^{t-1}\right) \quad \sigma \text{ is the activation function}$$

$$\frac{\partial \mathbf{h}^t}{\partial \mathbf{h}^{t-1}} = \operatorname{diag}\left(\sigma'\left(\mathbf{W}^t\mathbf{h}^{t-1}\right)\right)\left(\mathbf{W}^t\right)^T \quad \sigma' \text{ is the gradient function of } \sigma$$

$$\prod_{i=t}^{d-1} \frac{\partial \mathbf{h}^{i+1}}{\partial \mathbf{h}^i} = \prod_{i=t}^{d-1} \operatorname{diag}\left(\sigma'\left(\mathbf{W}^i\mathbf{h}^{i-1}\right)\right)\left(\mathbf{W}^i\right)^T$$

Gradient Exploding

- Use ReLU as the activation function

$$\sigma(x) = \max(0, x) \text{ and } \sigma'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Elements of $\prod_{i=t}^{d-1} \frac{\partial \mathbf{h}^{i+1}}{\partial \mathbf{h}^{i}} = \prod_{i=t}^{d-1} \operatorname{diag}\left(\sigma'\left(\mathbf{W}^{i}\mathbf{h}^{i-1}\right)\right)\left(\mathbf{W}^{i}\right)$ thay come from $\prod_{i=t}^{d-1} \left(\mathbf{W}^{i}\right)^{T}$

- Leads to large values when d-t is large

$$1.5^{100} \approx 4 \times 10^{17}$$

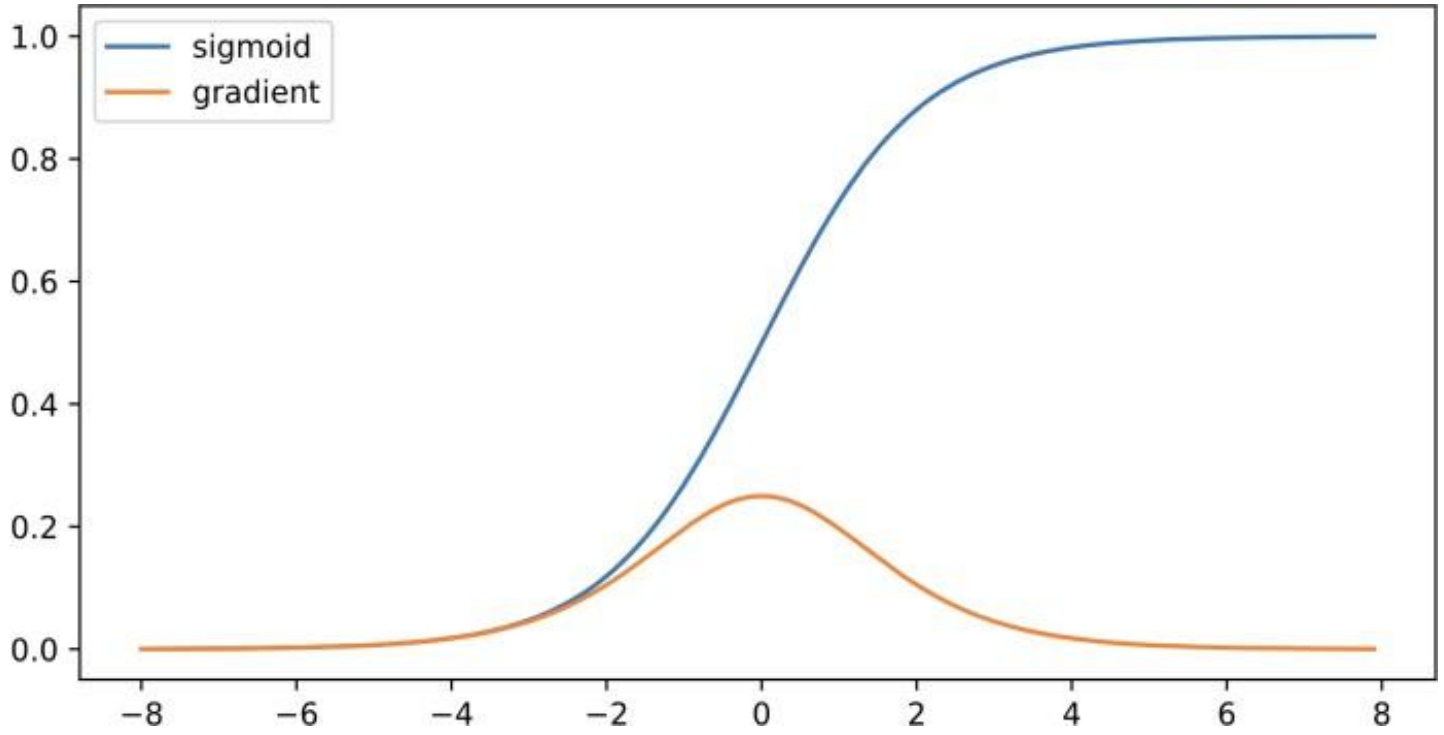
Issues with Gradient Exploding

- Value out of range: infinity value
  - Severe for using 16-bit floating points (6e-5 - 6e4)
- Sensitive to learning rate (LR)
  - Not small enough LR -> large weights -> larger gradients
  - Too small LR -> No progress
  - May need to change LR dramatically during training

Gradient Vanishing

- Use sigmoid as the activation function

$$\sigma(x) = \frac{1}{1+e^{-x}} \qquad \sigma'(x) = \sigma(x)(1-\sigma(x))$$



Gradient Vanishing

- Use sigmoid as the activation function

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

- Elements $\prod_{i=t}^{d-1} \frac{\partial \mathbf{h}^{i+1}}{\partial \mathbf{h}^{i}} = \prod_{i=t}^{d-1} \mathrm{diag}(\sigma'(\mathbf{W}^i \mathbf{h}^{i-1}))(\mathbf{W}^i)^T$ are products of *d-t small values.*

$$0.8^{100} \approx 2 \times 10^{-10}$$

Issues with Gradient Vanishing

- Gradients with value 0
  - Severe with 16-bit floating points (gradient less than $2^{-24} \approx 5.96 \times 10^{-8}$ is 0)
- No progress in training
  - No matter how to choose learning rate
- Severe with bottom layers
  - Only top layers are well trained
  - No benefit to make networks deeper

Summary

- Numbers too big or too small will lead to numerical problems
- Often happen in deep models, because it multiplies many numbers

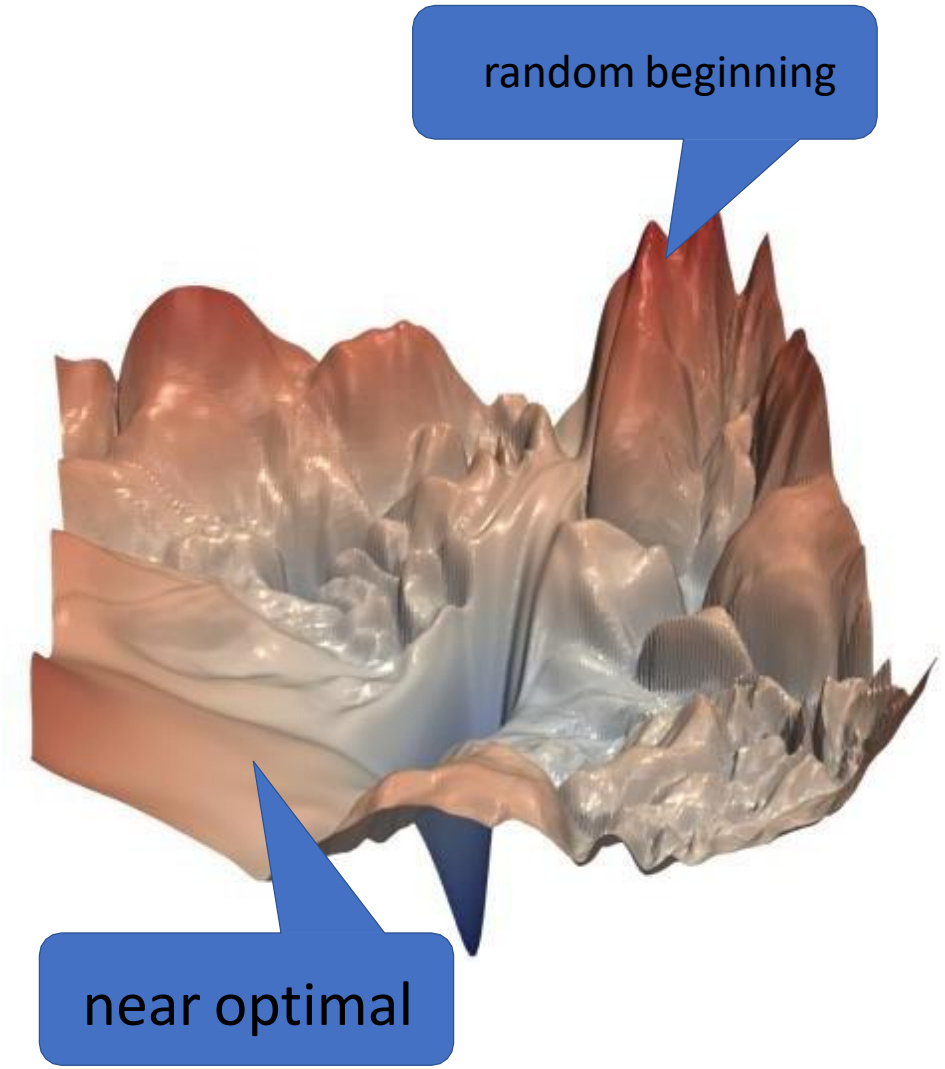
Stabilize Training

Stabilize Training

- Goal: make sure gradient values are in a proper range
  - E.g. in [1e-6, 1e3]
- Multiplication -> plus
  - ResNet, LSTM
- Proper weight initialization
- Batch normalization

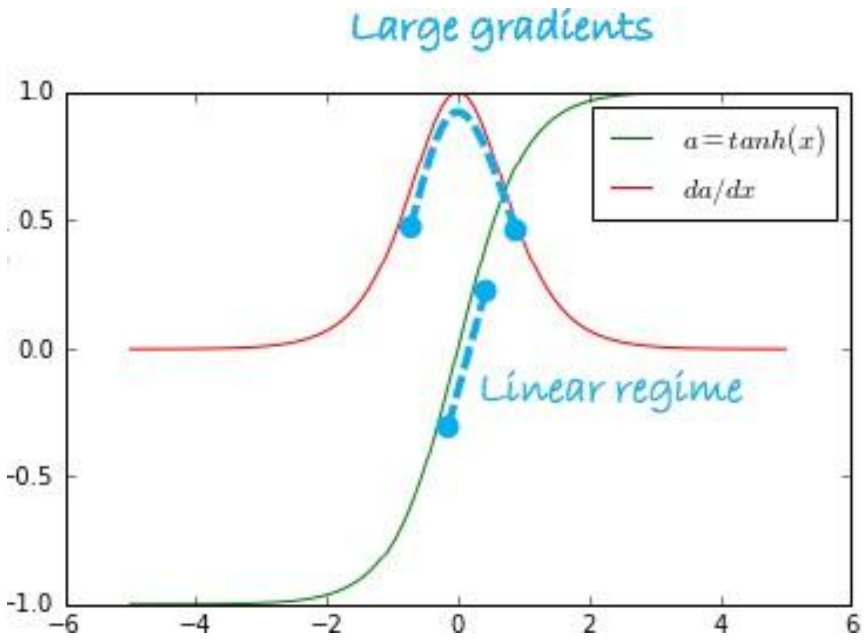
Stabilization 1: Weight Initialization

- Initialize weights with random values in a proper range
- The beginning of training easily suffers to numerical instability
  - The surface far away from an optimal can be complex
  - Near optimal may be flatter

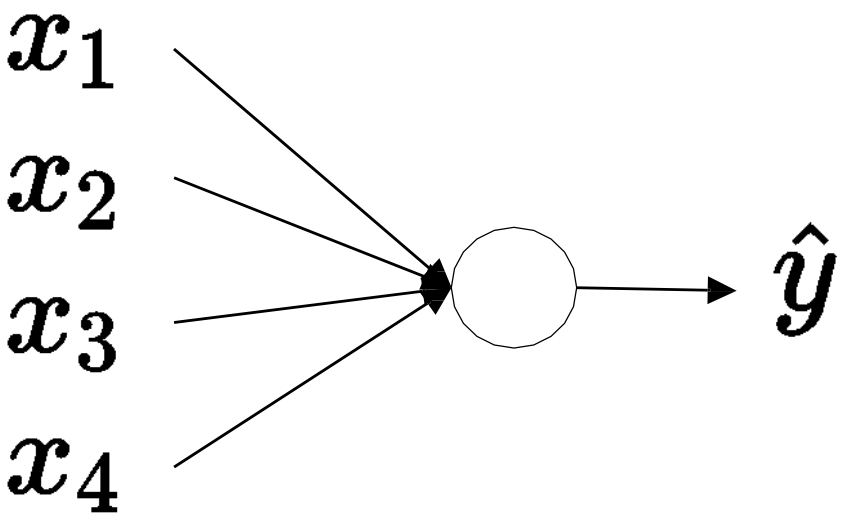


Random Initialization

- Weights need to be small enough
  - Training starts, better near linear regime
  - Larger gradients → faster training
- Weights need to be large enough
  - Otherwise, signal is too weak for any serious learning



Single neuron example



$$\mathrm{h} = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

$$\text{Large d} \rightarrow \text{ small w}$$

$$\mathrm{Var}(w_i) = \frac{1}{d}$$

Xavier initialization [Glorot2010]

- Draw random weights from

$$w \sim N\left(0, \sqrt{1/d}\right)$$

where d is the number of neurons in the input

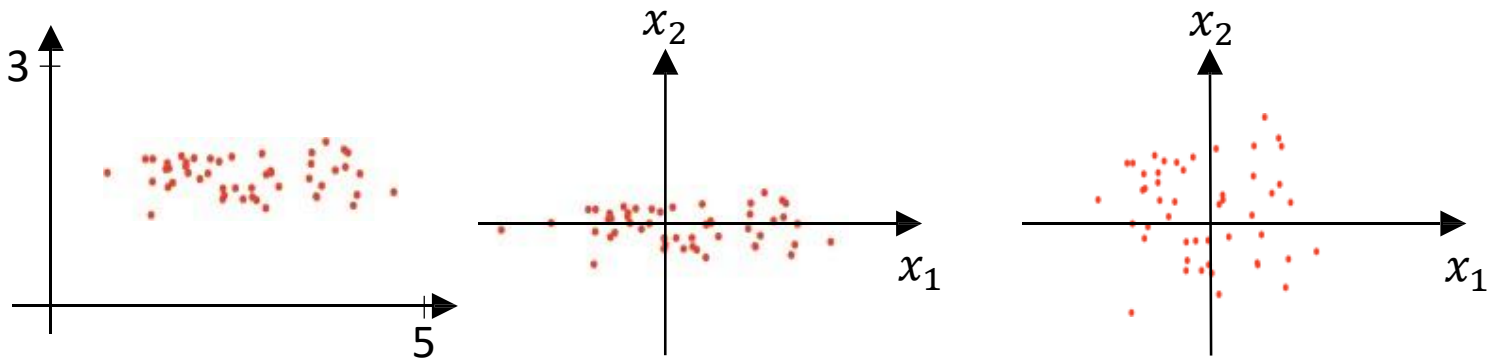
[He2015] initialization for ReLUs

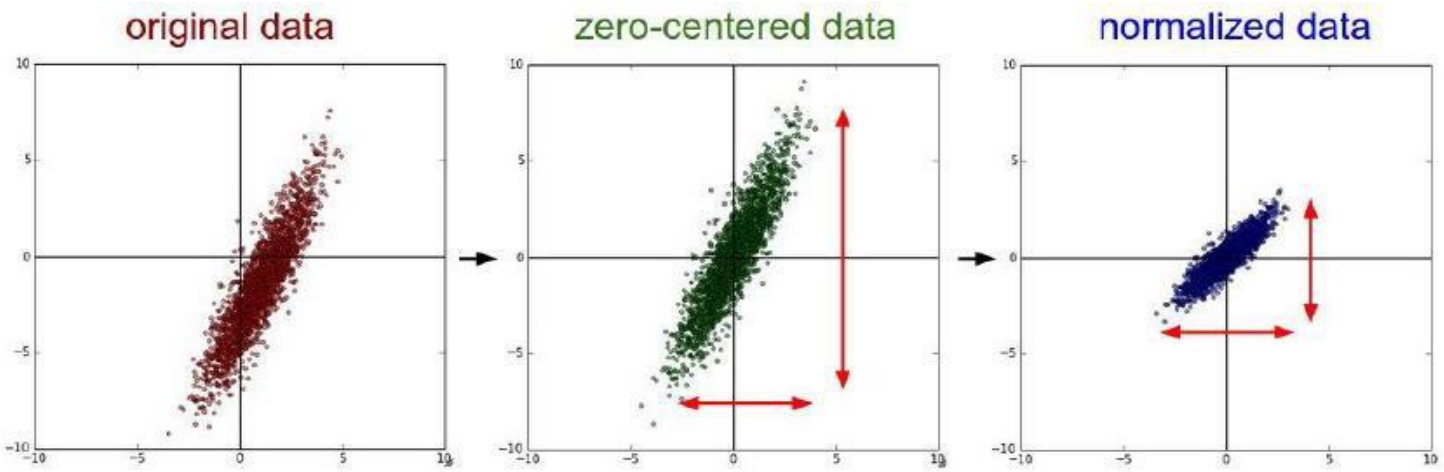
- Unlike sigmoids, ReLUs ground to 0 the linear activations half the time
- Draw random weights from $\mathrm{w} \sim N\left(0, \sqrt{2/d}\right)$

  where d is the number of neurons in the input

Stabilization 2: Input normalization

Normalizing training sets

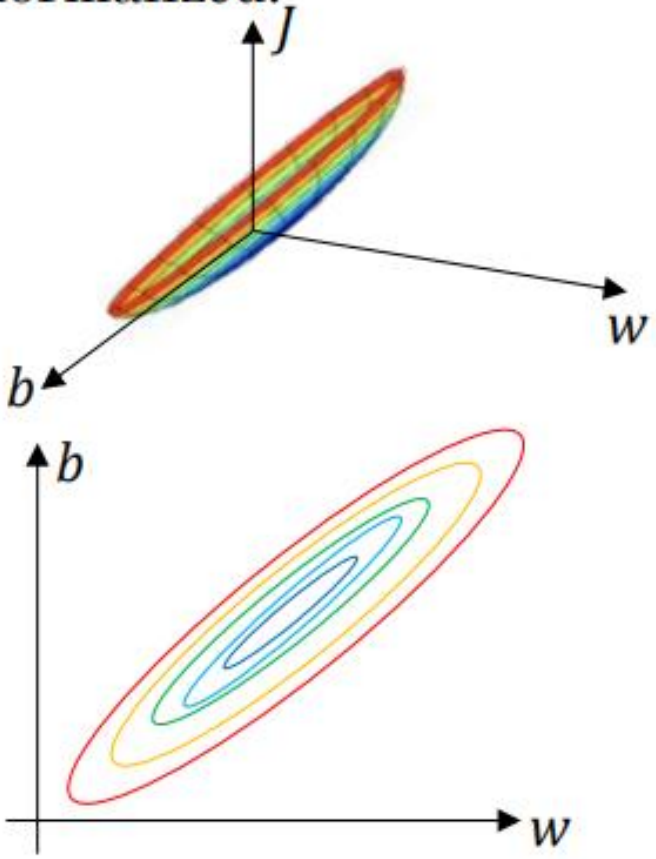




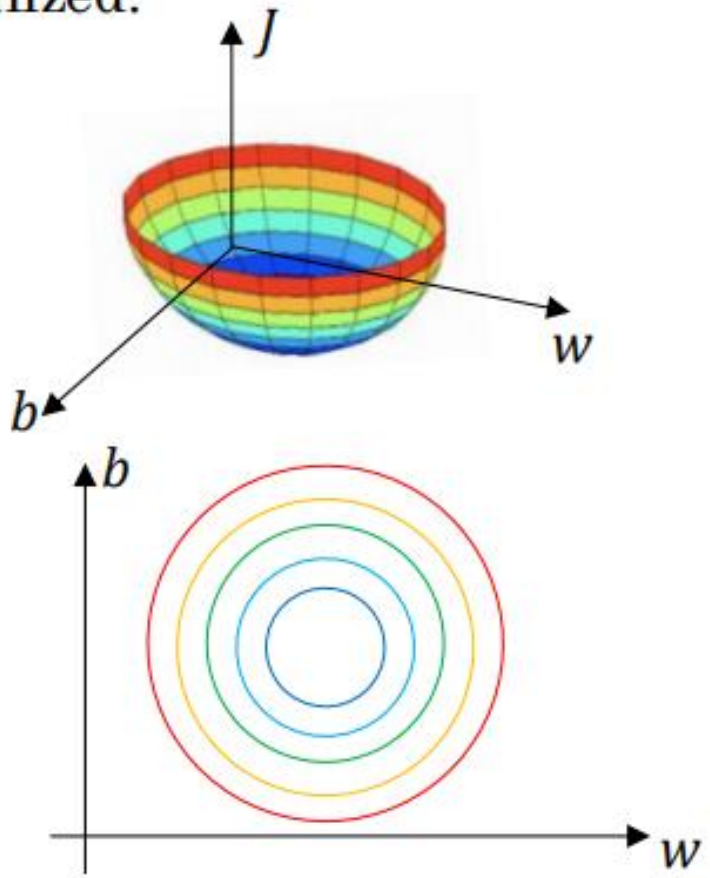
Why normalize inputs?

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right)$$

Unnormalized:



Normalized:



In practice for Images: center only

- High variance could be a good feature
  - A photo that is almost dark (low variance) is very likely to be in the night

Stabilization 3: Batch normalization

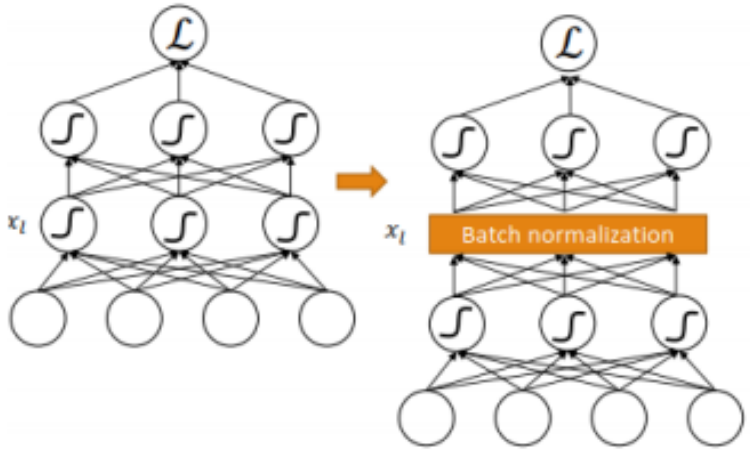
"you want zero-mean unit-variance activations? just make them so."

$$\mu = \frac{1}{m}\sum_i z^{(i)}$$ [compute mini-batch mean]

$$\sigma^2 = \frac{1}{m}\sum_i (z^{(i)} - \mu)^2$$ [compute mini-batch variance]

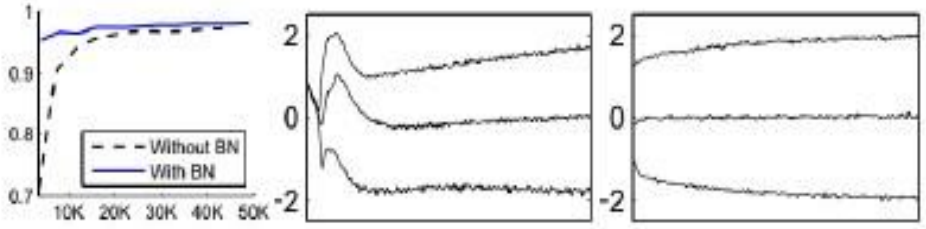
$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$ [normalize input]

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$ [scale and shift input]



Batch normalization -Benefits

- Gradients can be stronger → higher learning rates → faster training
  - Otherwise maybe exploding or vanishing gradients or getting stuck to local minima
- Neurons get activated in a near optimal "regime"
- Better model regularization
  - Neuron activations not deterministic, depend on the batch
  - Model cannot be overconfident
- Acts as a regularizer
  - The per mini-batch mean and variance are a noisy version of the true mean and variance
  - Injected noise reduces overfitting during search



(a) (b) Without BN (c) With BN

Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as* $\{15, 50, 85\}$*th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

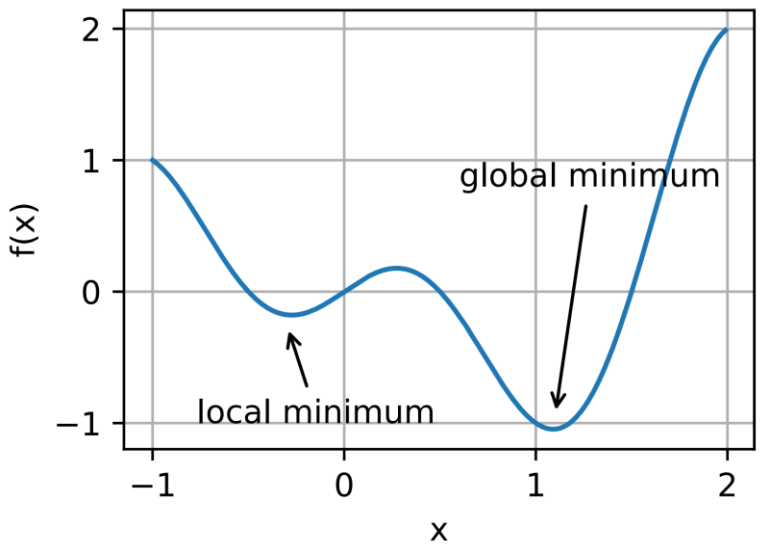
code

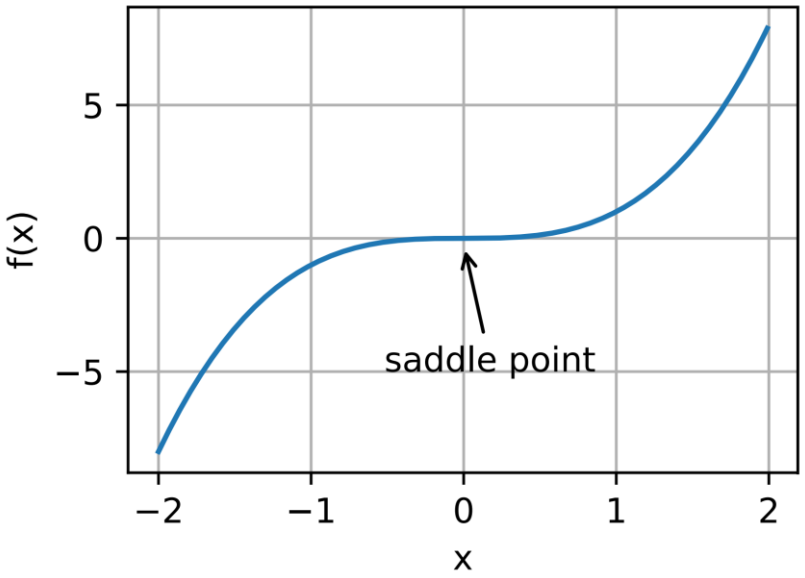
Add batch normalization after linear layer and before activation function

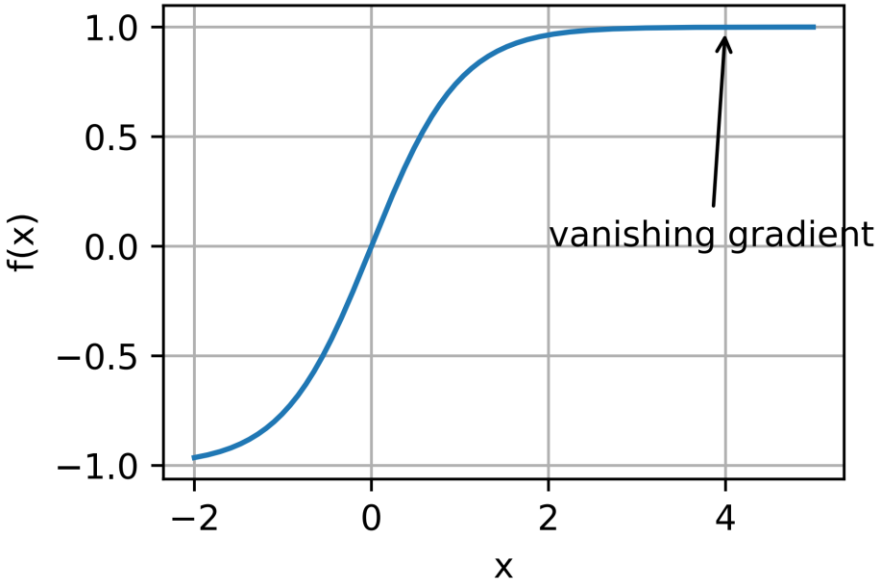
```
net = nn.Sequential(nn.Linear(d,256),nn.BatchNorm1d(256), nn.ReLU())
```

Advanced Optimization

Optimization Challenges

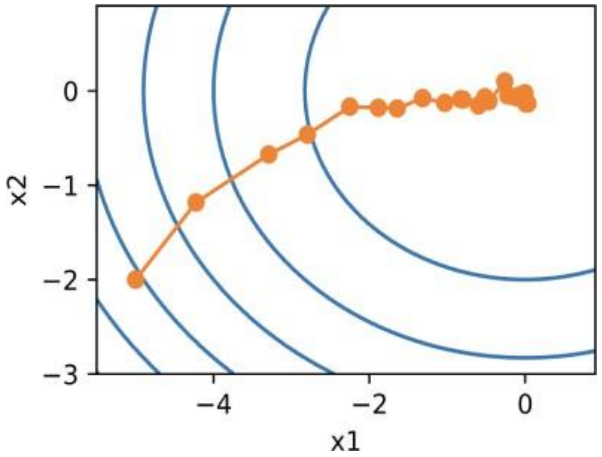




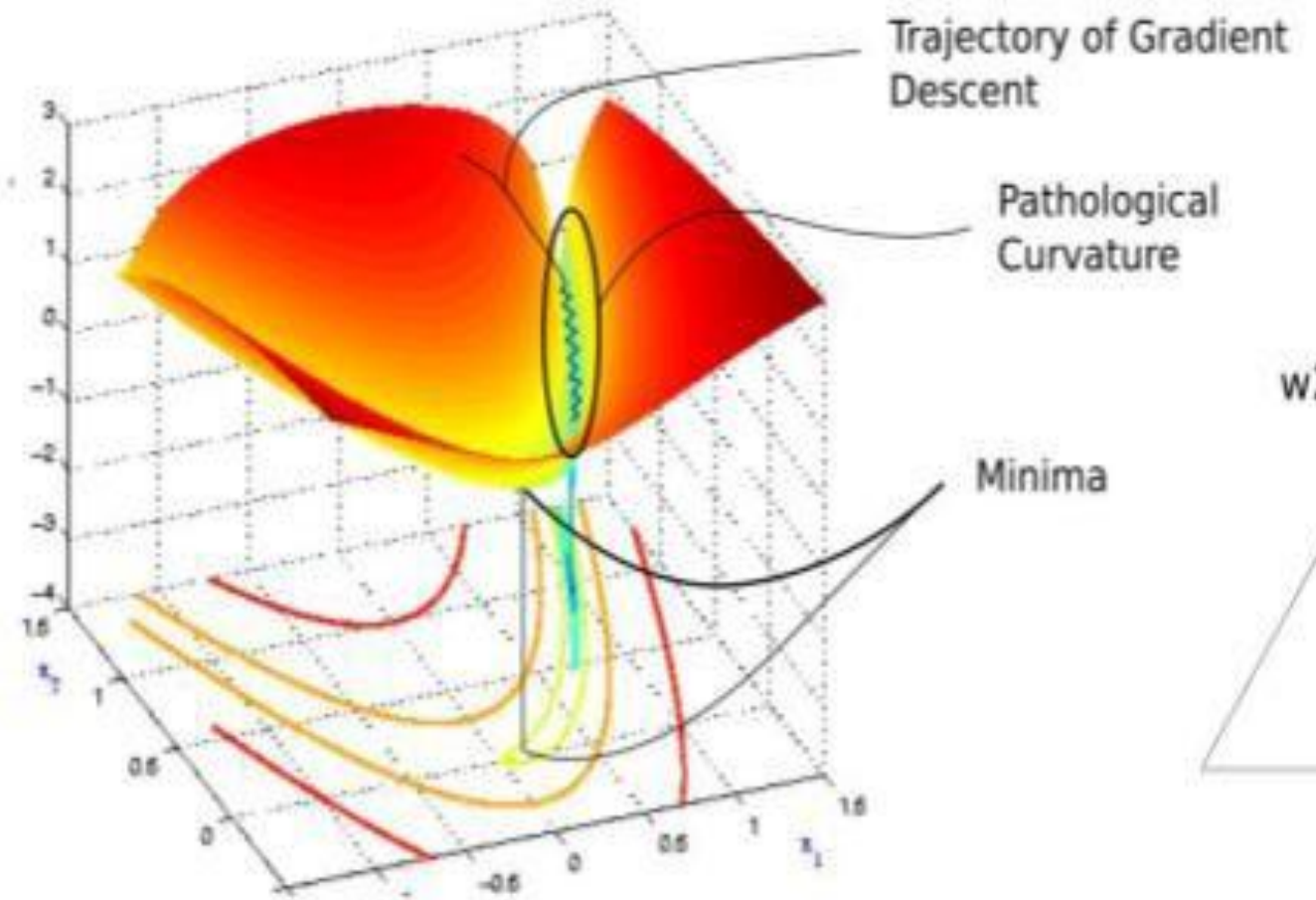


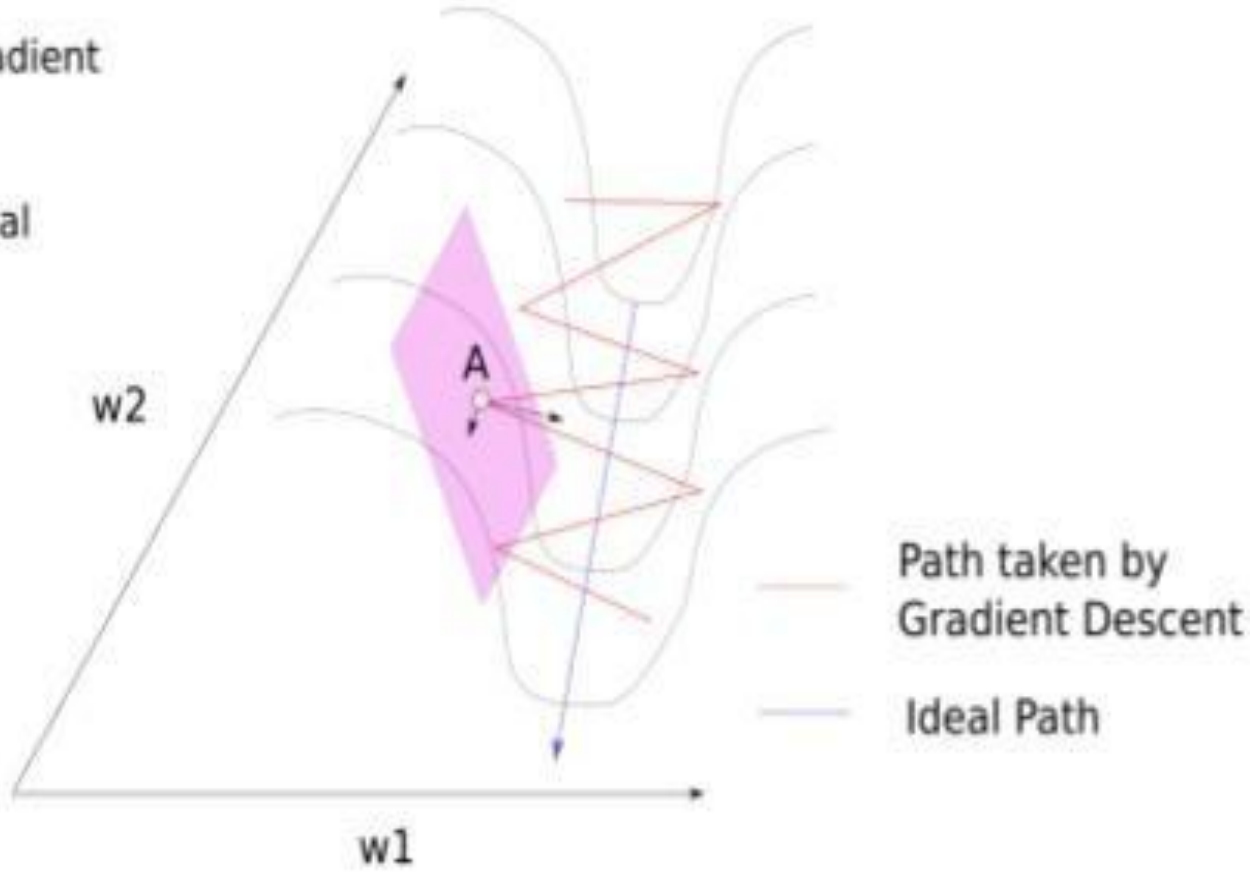
Mini-batch SGD

- Computing the gradient over the whole training data is too expensive
  - Takes hours for DNN models
- Randomly sample *m* examples to approximate the loss:
- The empirical loss for the whole dataset with N data points: $\frac{1}{N}\sum_{i=1}^{N} l(\boldsymbol{x}_i, y_i, \boldsymbol{w})$
- Loss in a mini-batch: $\frac{1}{m}\sum_{i=1}^{m} l(\boldsymbol{x}_i, y_i, \boldsymbol{w})$



Motivation

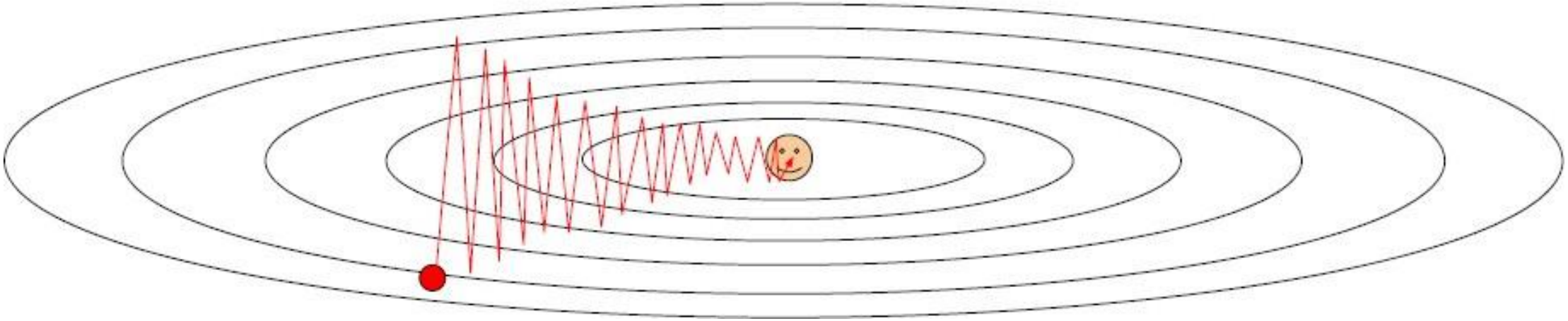




Picture credit: Team Paperspace

Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction

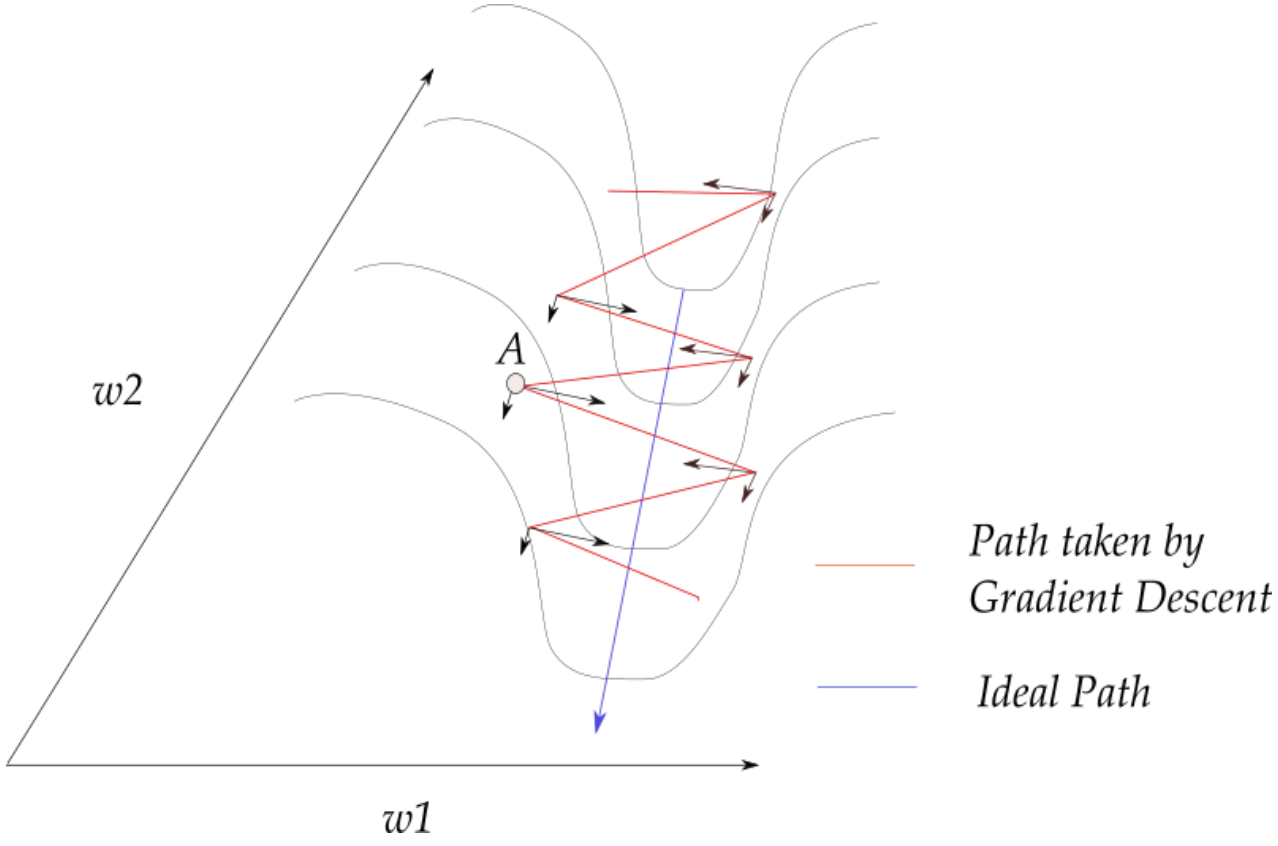


Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

Gradient descent with momentum

- Don't switch gradients all the time
- Maintain "momentum" from previous parameters → dampens oscillations
- The exponential averaging cancels out the oscillating gradients
- More robust gradients and learning → faster convergence



Gradient descent with momentum

On iteration t:

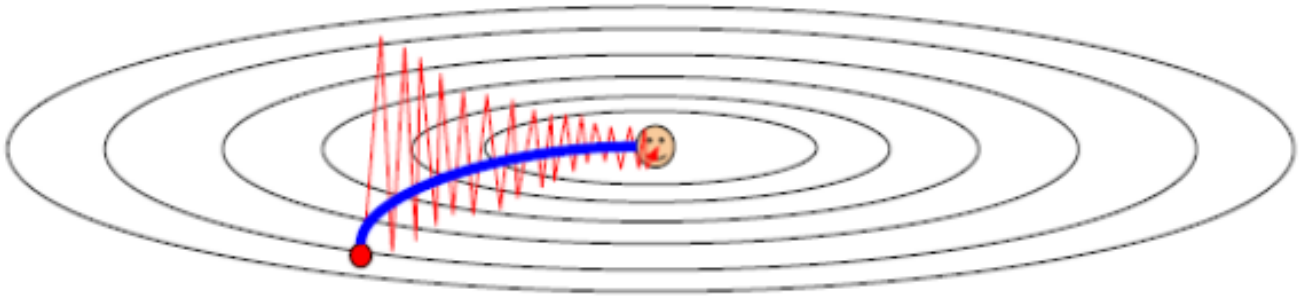
Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1-\beta)dW$$

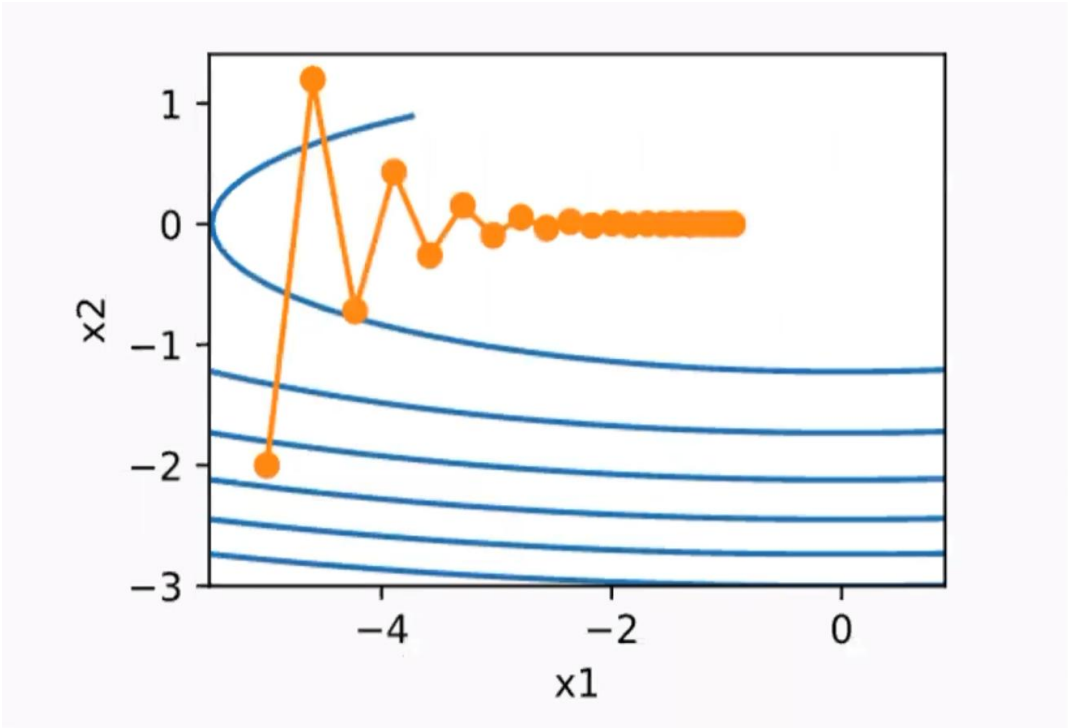
$$v_{db} = \beta v_{db} + (1-\beta)db$$

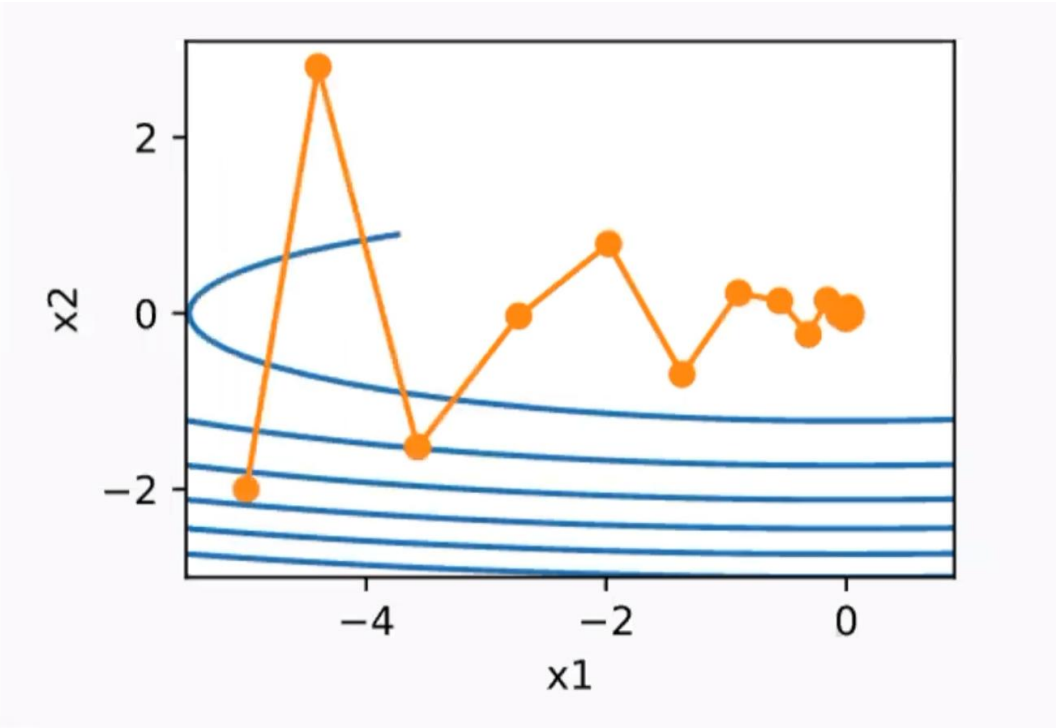
$$W = W - \alpha v_{dW},\ b = b - \alpha v_{db}$$

Hyperparameters: α, β $\qquad \beta = 0.9$



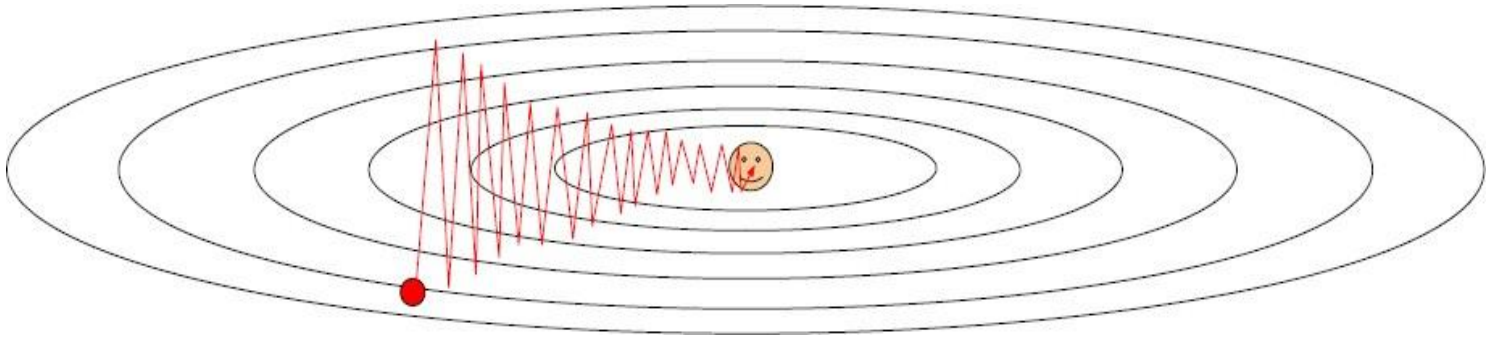
Improvement





Common choices of β: $0.5, 0.9, 0.95, 0.99$

RMSprop



- **Large gradients**, e.g. too "noisy" loss surface
  - Updates are tamed
- **Small gradients**, e.g. stuck in flat loss surface ravine
  - Updates become more aggressive

One parameter (learning rate) may not be enough to control the step length at all directions. And we don't even know which parameter is updated with significant changes.

Need some self-adapted method to make sure all w are updated in a roughly same rate.

RMSprop

For each parameter w and b, add a new parameter S called scale

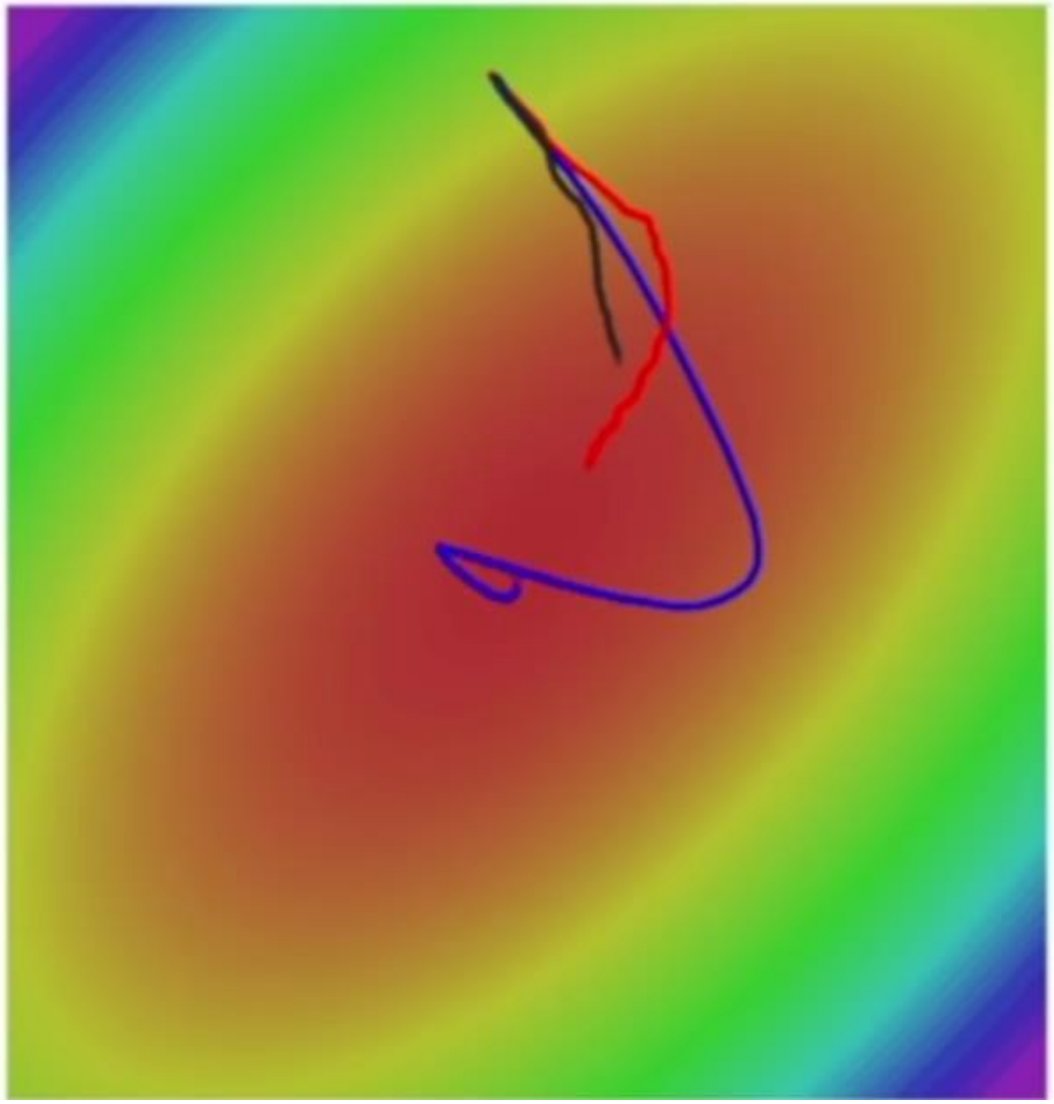
On iteration t:

Compute dW, db on the current mini-batch

$$S_{dW} = \beta S_{dW} + (1-\beta) dW^2$$

$$S_{db} = \beta S_{db} + (1-\beta) db^2$$

$$W = W - \alpha \frac{dW}{\sqrt{S_{dW}}} \qquad b = b - \alpha \frac{db}{\sqrt{S_{db}}}$$



SGD

SGD+Momentum

RMSProp

Adam optimization algorithm [Kingma2014]

Compute Gradient dW_t

$$v_t = \beta_1 v_{t-1} + (1-\beta_1) dW_t$$

$$S_t = \beta_2 S_{t-1} + (1-\beta_2) dW_t^2$$

$$w_t = w_{t-1} - \eta \frac{v_t}{\sqrt{S_t}}$$

Adaptive Moment Estimation

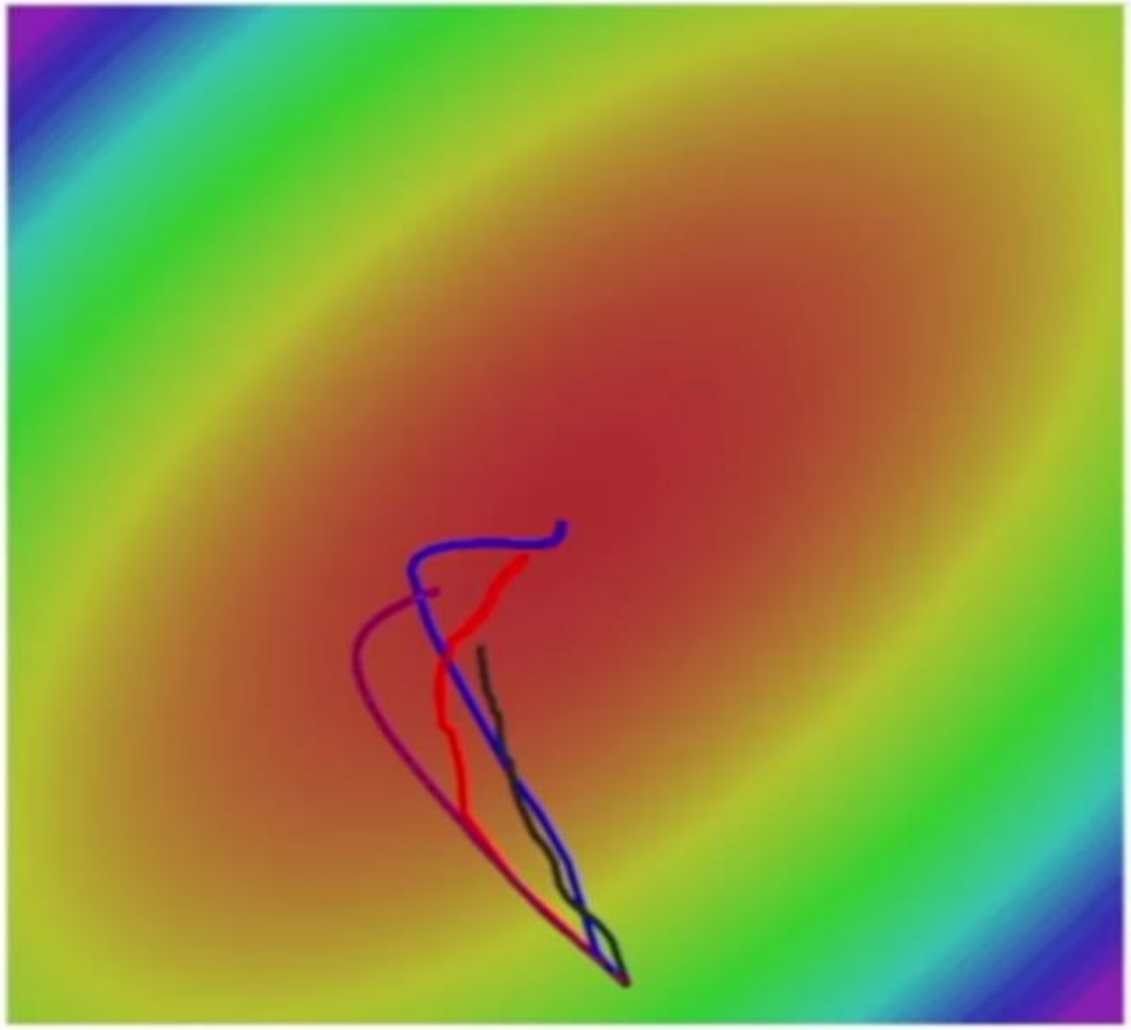
Hyperparameters choice:

α: needs to be tune

$\beta = 0.9$: momentum (dW)

$\beta = 0.999$: RMSprop (dW^2)

- Similar to RMSprop, but with momentum & correction bias



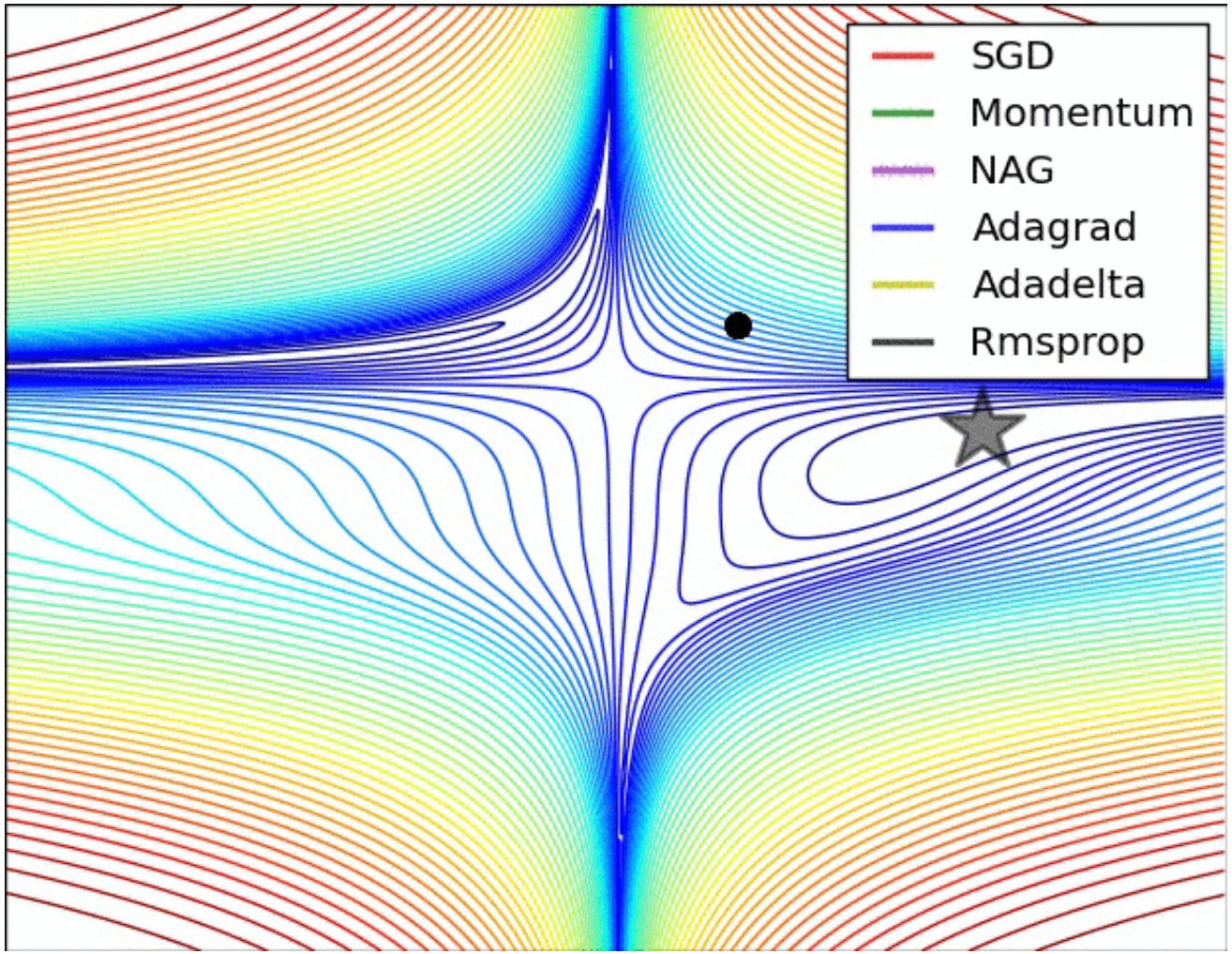
SGD

SGD+Momentum

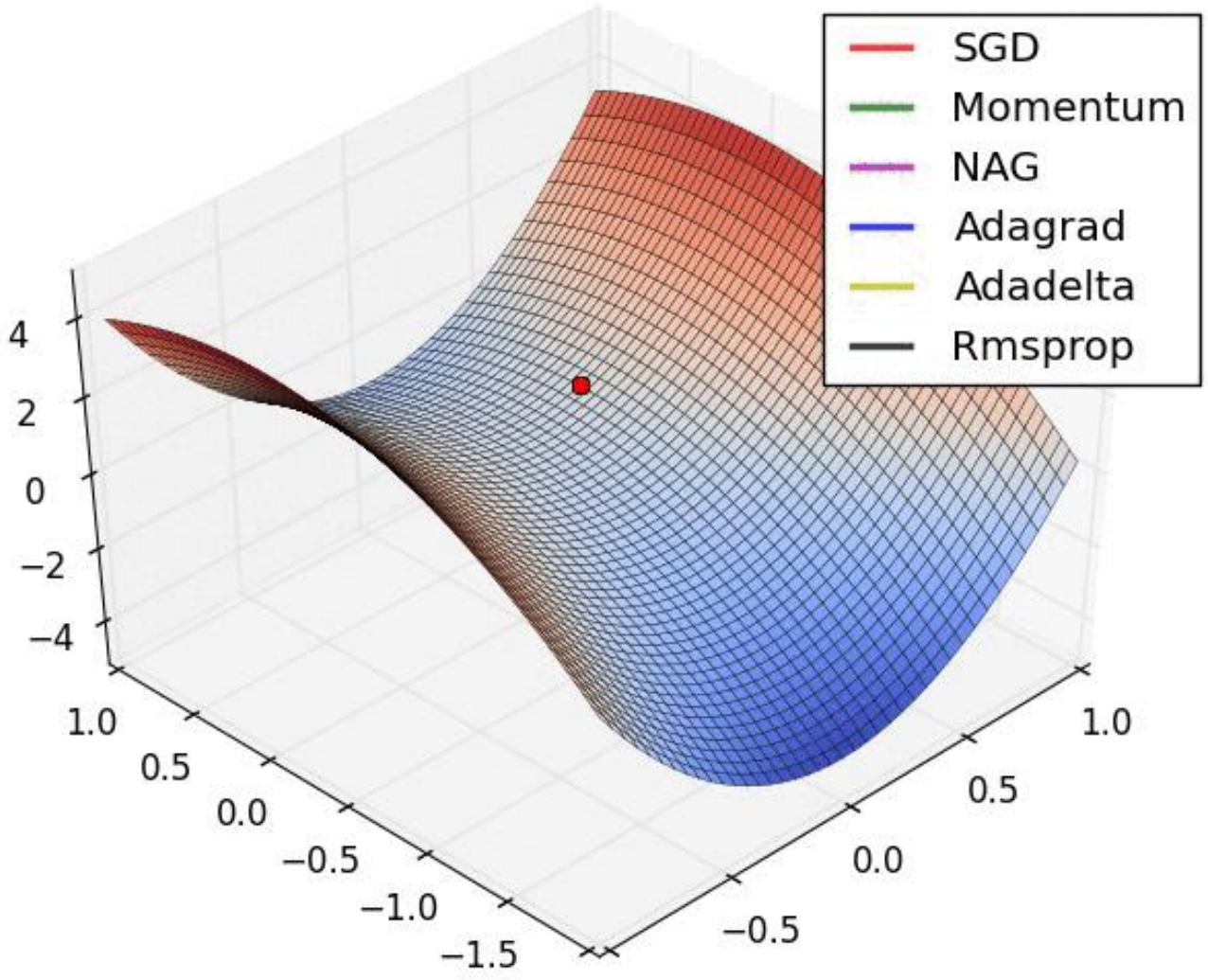
RMSProp

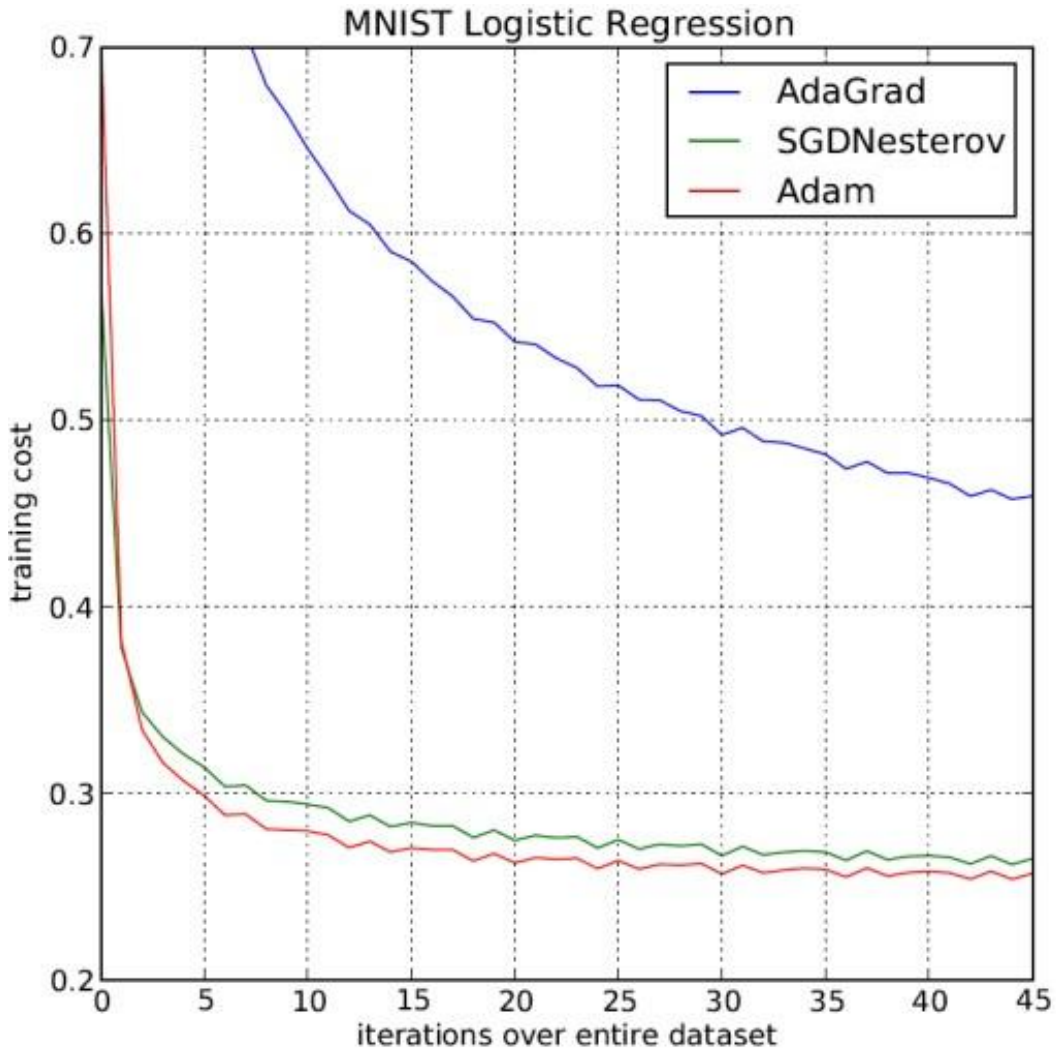
Adam

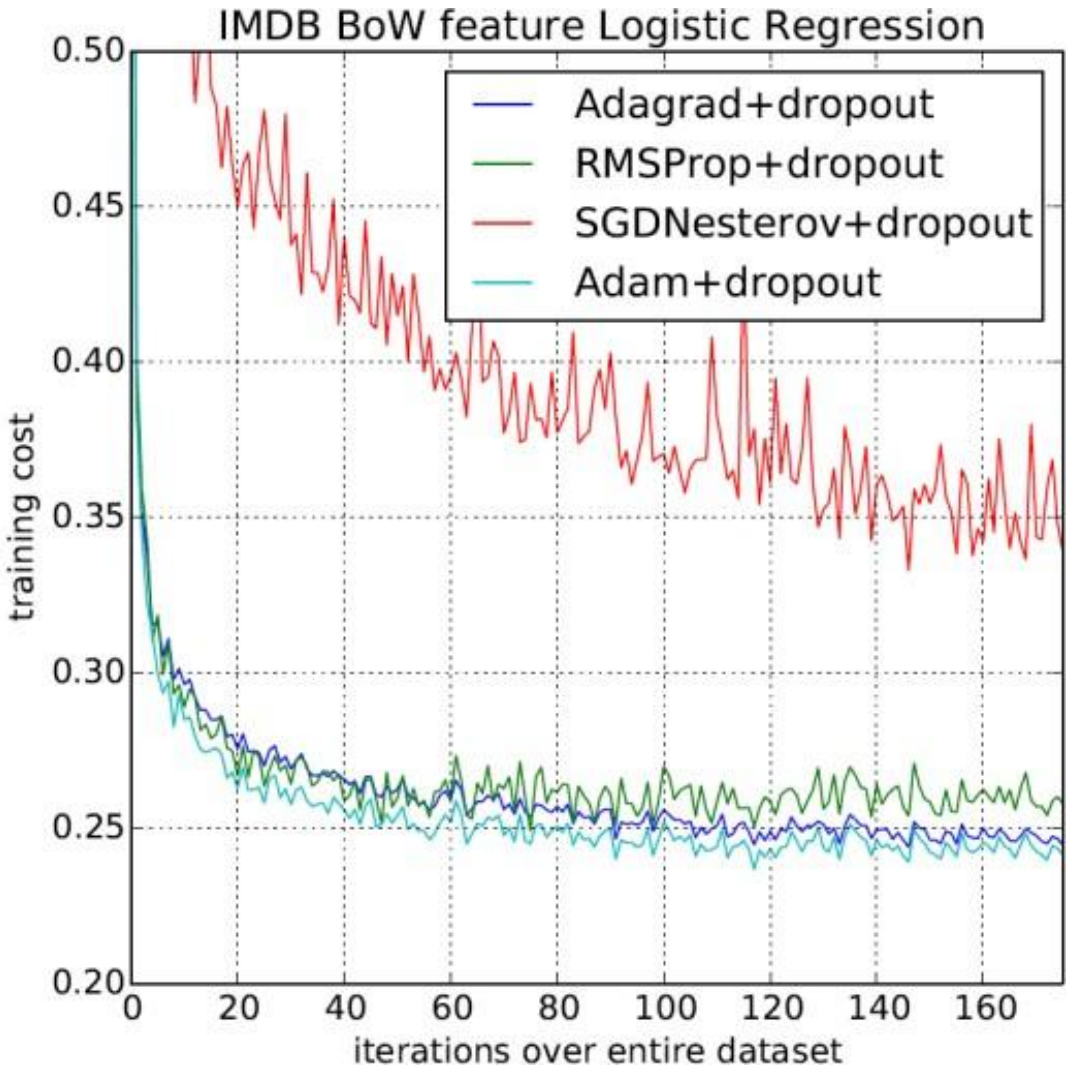
SGD optimization on loss surface contours



SGD optimization on saddle point




MNIST Logistic Regression
AdaGrad
SGDNesterov
Adam
training cost
iterations over entire dataset
0.7
0.6
0.5
0.4
0.3
0.2
0
5
10
15
20
25
30
35
40
45


IMDB BoW feature Logistic Regression
Adagrad+dropout
RMSProp+dropout
SGDNesterov+dropout
Adam+dropout
training cost
iterations over entire dataset
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0
20
40
60
80
100
120
140
160

```
torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
torch.optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)
torch.optim.Adam(model.parameters(), lr=0.01, betas=(0.9, 0.999))
```

Learning rate decay